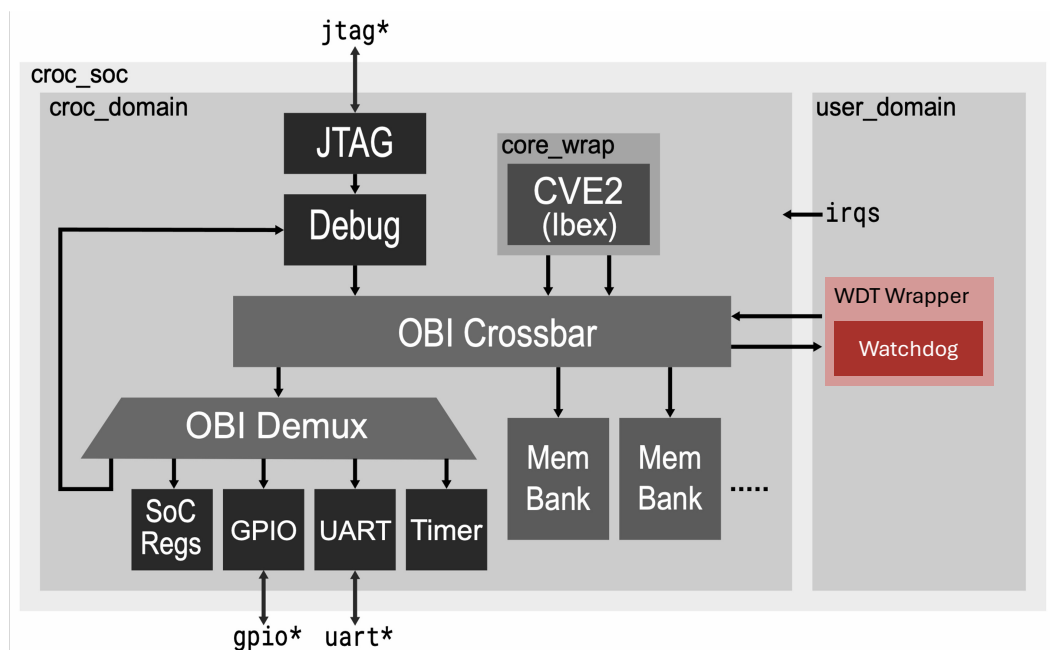


DEPARTMENT OF INFORMATION TECHNOLOGY AND
ELECTRICAL ENGINEERING

Spring Semester 2025

Extending CROC SoC with a Hardware Watchdog

VLSI 2 Course project

Miguel Correa and Elio Wanner
corream@ethz.ch, ewanner@ethz.ch

June 12, 2025

Professor: Frank Kagan Gürkaynak

Contents

1	Introduction	1
1.1	Overview of the Croc SoC	1
1.2	Motivation for Watchdog Integration	1
1.3	Project Objectives	1
2	Related Works	3
2.1	Overview of Existing Watchdog Implementations	3
2.2	Justification for our approach	3
3	Methodology	4
3.1	Design choices for our watchdog	4
3.2	Debugging, testing and iteration process	4
4	Hardware Architecture	6
4.1	Simple Watchdog	6
4.1.1	Hardware implementation	6
4.1.2	Tests and Results	7
4.2	Watchdog Wrapper	8
4.2.1	Hardware Implementation	8
4.2.2	Tests and Results	8
5	Evaluation	9
5.1	Simulation results	9
5.2	Assessment of Project Objectives	9
5.3	Possible improvements	9
6	Conclusion	11
6.1	Summary of key findings	11
6.2	Future Work and Next Steps	11

Introduction

1.1 Overview of the Croc SoC

CROC SoC is a lightweight, open-source system-on-chip designed for education and rapid prototyping. It provides a complete RTL-to-GDSII flow and basic processor and peripheral infrastructure, and even includes an explicit user_domain where students can add and test their own modifications. As a relatively new project, CROC is not yet fully optimized, leaving plenty of room to enhance its reliability, performance, and set of features. [1]

1.2 Motivation for Watchdog Integration

In May 1994, the NASA Clementine lunar probe suffered a critical onboard computer failure shortly after departing lunar orbit. A malfunction of the on-board computer caused one of its thrusters to fire uninterrupted, spinning the spacecraft at 80 rpm until it exhausted its remaining fuel. [2] This failure could have possibly been prevented by a hardware watchdog monitoring the onboard computer, which could have detected the malfunction and triggered a fast reset so that the thruster would have stopped to fire continuously.

On CROC SoC where students routinely extend the design in the course VLSI 2 and need to test custom peripherals, it can happen that the chip deadlocks or that it is found in an unusable state. Embedding a hardware watchdog ensures that such faults trigger an automatic reset to a known-working state, improving system robustness during development and testing after tape-out.

1.3 Project Objectives

This project aims to design and integrate a simple hardware watchdog into the CROC SoC, with the following main goals:

- **Reliable Fault Detection:** Detect processor stalls or bus deadlocks within a bounded time-out period.
- **Automatic Recovery:** Trigger a chip-wide reset when the watchdog expires, returning the system to a known working state.

- **Minimal Overhead:** Add the watchdog with low area, timing, and power impact, preserving CROC's lightweight footprint.

Furthermore, we were given an area constraint of $2235 \times 2235 \mu\text{m}$. Taking into account a searing of $50 \mu\text{m}$ and leaving $70 \mu\text{m}$ for the bonding pads on each side, this results in an effective area of $1995 \times 1995 \mu\text{m}$ available for logic placement.

With these improvements, the watchdog will increase CROC's robustness without losing its overall simplicity.

Related Works

2.1 Overview of Existing Watchdog Implementations

Several types of watchdogs exist, each offering different levels of complexity and fault tolerance. They can be implemented either on-chip or as an external hardware module or then in software. Here is a non-exhaustive list:

- **Single-stage watchdog:** A simple counter that resets the system if not cleared in time. [3]
- **Multistage watchdog:** First stage may raise an interrupt; final stage triggers a reset. Allows for optional error handling before reboot. [3]
- **Software watchdog:** Same as the single-stage hardware watchdog, but implemented in software and therefore not super reliable. [4]

A watchdog can be implemented with varying levels of complexity. In its simplest form, it resets the system after a fixed timeout (single-stage watchdog). A more advanced version first sends a warning signal to the processor before resetting, allowing it to attempt recovery. A thorough implementation may also collect log data during the reset to support post-reset debugging.

2.2 Justification for our approach

We chose a simple single-stage hardware watchdog that issues a system reset after a timeout. This approach minimizes area and power overhead, aligns with the lightweight nature of the Croc SoC, and ensures that the module is easy to understand, verify, and repurpose if needed. By avoiding features such as logging or staged resets, we keep complexity low while still improving the system robustness. The watchdog is integrated in the user domain and communicates with the core via OBI, making it non-intrusive yet effective. This trade-off offers a good balance between added reliability and minimal design impact.

In this way, we will have the reliability and robustness benefits of adding a hardware watchdog without having a big overhead in terms of power or area.

We provide a lightweight software runtime library for setting the timeout and sending the periodic kick signal.

Methodology

This chapter explains the development process, the tools that were used, and the choices we made along the way.

3.1 Design choices for our watchdog

To equip CROC with automatic fault recovery, we designed a hardware watchdog and an OBI-protocol wrapper in the `user_domain`. Our key design decisions were as follows:

- **Count-up timer:** We adopted the classic count-up approach (versus count-down) because it aligns with industry practice and synthesizes to minimal logic. On each clock cycle that the watchdog is enabled and not refreshed, a small counter increments. Reaching its terminal value triggers a timeout event.
- **One-cycle reset pulse:** Upon timeout, the watchdog emits a one-cycle reset pulse. One cycle is sufficient to assert the global reset input across all domains and restart the entire SoC from its defined reset vector, without introducing undue latency or complicating the reset tree.
- **Hardware implementation:** A hardware watchdog can detect and recover from processor hangs or bus deadlocks even when the CPU itself has stopped executing instructions. A purely software watchdog (for instance implemented as a loop in firmware), cannot fire if the CPU is already stalled, so it cannot guarantee recovery in all fault scenarios.
- **OBI-protocol wrapper:** The wrapper translates OBI reads and writes into the internal control signals of the counter, keeping the watchdog core itself free of bus-interface complexity.
- **Placement in `user_domain`:** By locating the watchdog in the `user_domain`, we avoid touching the `croc_domain`'s stable, taped-out infrastructure.

3.2 Debugging, testing and iteration process

We approached development in two separate stages, making debugging manageable and systematic. Initially, we focused on building and verifying the standalone watchdog module. One of us

3 Methodology

implemented the watchdog logic while the other developed a testbench to verify its correctness. Secondly we integrated the watchdog within CROC.

Our testbench included both typical use cases and edge cases, such as issuing a kick signal at the last possible moment or kicking the watchdog continuously. This ensured the counter reset logic and timeout behavior were robust and as expected. A custom Makefile automated the process: it compiled the design, generated the stimuli, ran simulations, and checked the outputs.

We used an all open-source toolchain as introduced in the course. Verilator was our primary tool for RTL simulation, which allowed us to observe signal-level behavior and validate functionality. Yosys was used for logic synthesis, and OpenROAD helped in the physical design steps later in the flow.

Once the standalone watchdog was verified, we moved on to integration with the CROC SoC. This added considerable complexity and required building two additional components:

1. A hardware wrapper to interface the watchdog with the OBI bus protocol
2. A lightweight software runtime to configure the timeout and issue periodic kick signals.

The wrapper module went through several iterations. Debugging this was a bit more tedious as we had to follow the given OBI protocol. We first focused on just getting it to run and then in a last iteration simplified it and cleaned the code. In this step, having the vcd file with the waveform was very helpful.

Hardware Architecture

This chapter describes the hardware architecture of the watchdog that we implemented in the Croc SoC. In the different sections we talk about different implementations with different features and complexities. In Figure 4.1 the whole system with the watchdog in the user domain is seen. It is solely connected to the rest of the system through its wrapper to the OBI Crossbar.

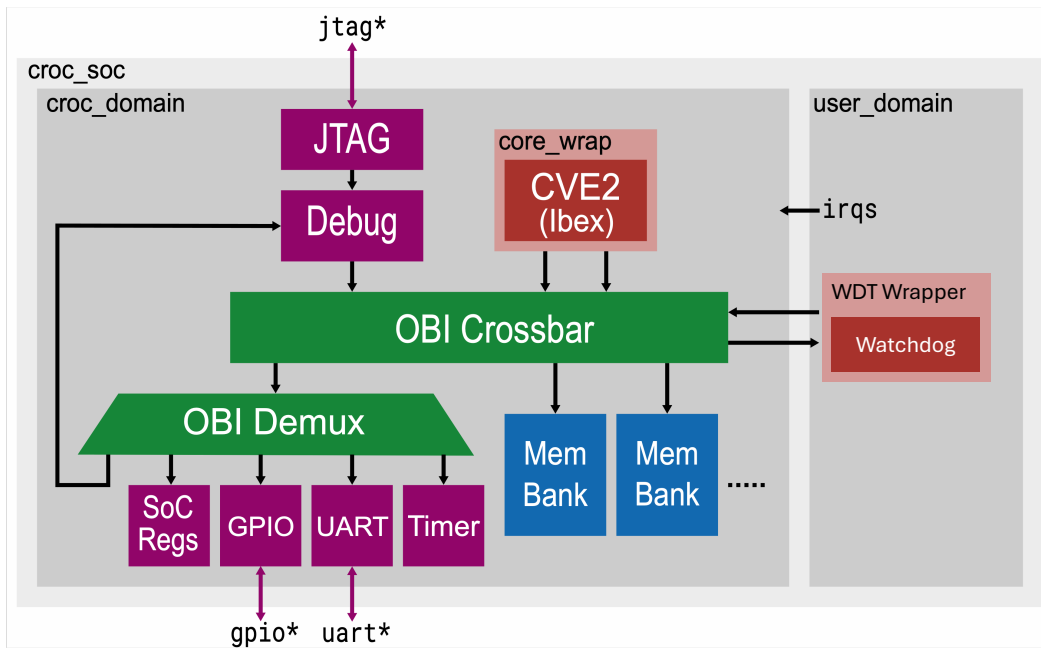


Figure 4.1: Croc Architecture with watchdog

4.1 Simple Watchdog

4.1.1 Hardware implementation

The first implementation of the watchdog is the simplest one we could imagine. As shown in Figure 4.2, the watchdog is composed of very few components. The main component is the counter, which is incremented every clock cycle and implemented as a flip-flop. The counter is

connected to a comparator that checks if the counter has reached or exceeded a specified timeout value. If the counter is greater than or equal to this value, the comparator triggers a reset signal that resets the system. Otherwise, the counter continues incrementing. As soon as a kick signal is received, the counter is reset to zero.

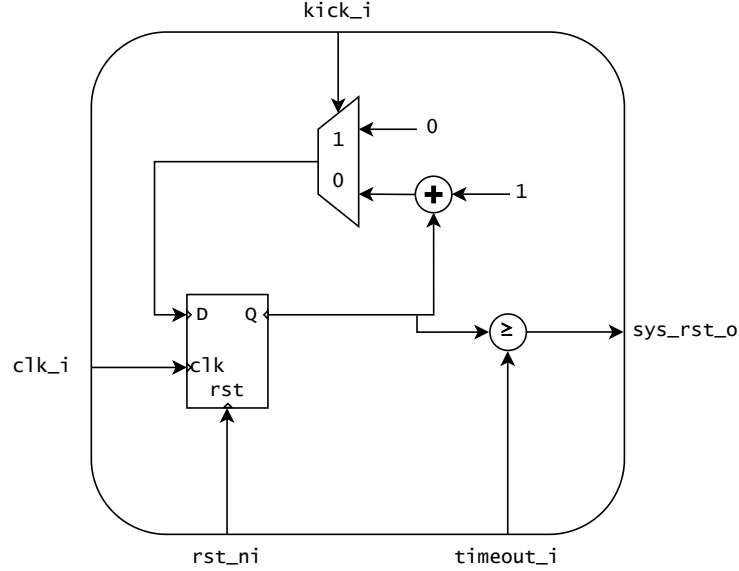


Figure 4.2: Simple Watchdog Architecture

4.1.2 Tests and Results

To verify the correctness of our standalone watchdog implementation, we created an automated test framework consisting of two Python scripts and a SystemVerilog testbench.

The script `gen_stimuli.py` generated random test vectors by selecting a timeout value and a cycle at which a kick signal is applied. The second script, `gen_exp_responses.py`, computed the expected behavior of the watchdog given the input stimuli. It evaluated whether a reset should occur and at which cycle.

These vectors were loaded into a SystemVerilog testbench, which applied the stimuli to the watchdog and recorded the actual behavior. The testbench then compared the observed reset output and timing with the expected values. Each test case was printed as either **Pass** or **Mismatch**, with detailed debug information in the event of an error.

Figure 4.3 shows a waveform from two representative test cases. In the first case, the timeout is set to 7 cycles. A kick is issued after 6 rising edges, which resets the counter in time and prevents a system reset. In the second case, the timeout is set to 4 cycles, but the kick only arrives at the 4th rising edge. Since the watchdog triggers a reset when the cycle count is greater than or equal to the timeout, a reset is correctly issued in this case.

In total, all 32 randomly generated test cases passed successfully, confirming the correct operation of our watchdog across a range of input scenarios. From this we can conclude that the watchdog itself works fine and that if a future problem rises, it will stem from a different part of the project.

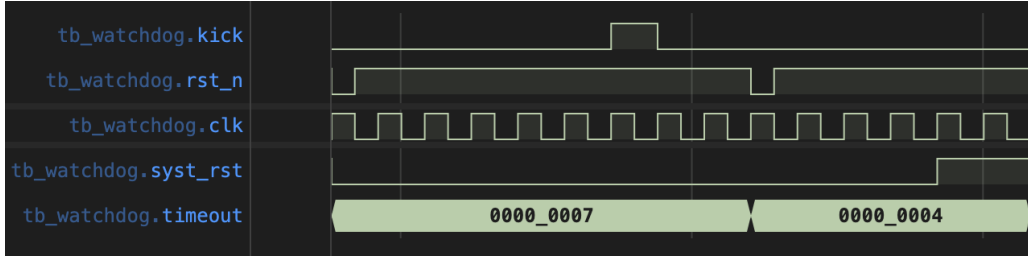


Figure 4.3: Snippet of the waveform from testing

4.2 Watchdog Wrapper

4.2.1 Hardware Implementation

To interface the standalone watchdog with the CROC SoC, we developed a wrapper module that translates memory-mapped OBI transactions into internal control signals for the watchdog. This wrapper enables software running on the SoC to configure the watchdog timeout and issue periodic kicks via standard memory accesses.

The wrapper decodes two 32-bit registers, from the base address 0x20001000 (address 0x20000000 is used as a ROM to save our names):

- **Address Offset 0x00:** Write-only register to issue a kick to the watchdog.
- **Address Offset 0x04:** Read-write register to set or read the timeout value.

To process these accesses, the wrapper stores request fields such as address, write enable, and write data in registers. On a valid write to address 0x00, it generates a one-cycle pulse to the `kick_i` input of the core watchdog. A write to 0x04 updates the internal timeout register, while a read from this address returns the currently stored timeout value. Invalid or unsupported transactions return default data with the error bit set.

The wrapper also captures the watchdog's `syst_rst_o` output and adds a one-cycle delay to allow software or hardware to observe the reset event in simulations.

The module was implemented with a focus on simplicity and protocol compliance. All OBI handshake and response signals are correctly managed, making the wrapper a fully compliant and lightweight interface to the watchdog.

4.2.2 Tests and Results

To verify the correct operation of the watchdog wrapper, we ran a test program `helloworld.c` that sets the timeout value and sends periodic kicks to prevent a reset. Toward the end of the program, the processor enters an infinite loop without issuing further kicks.

As expected, the watchdog triggered a system reset once the timeout was reached, confirming that the wrapper correctly propagates configuration and control signals to the core watchdog module.

Additionally, we inspected the VCD waveform to ensure the signal gets transferred correctly to the watchdog module. All handshake signals, response validity, and timing behavior were as expected, and no anomalies were observed.

Evaluation

5.1 Simulation results

Our watchdog passed all our simulations. It manages to restarts the chip when it is stuck in a task. Additionally, the final design passes both DRC and LVS checks with zero violations. This confirms the correctness of the physical implementation.

5.2 Assessment of Project Objectives

Our main objective, as described in Section 1.3, was to increase the reliability and robustness of the CROC SoC with minimal overhead. We can confidently say that this goal has been achieved.

The hardware watchdog successfully detects deadlocks and triggers a system-wide reset, ensuring recovery. This makes the SoC significantly more robust during development and testing.

The watchdog was implemented with minimal design complexity. It uses a simple counter and comparator logic, resulting in a very low area and power footprint. The area of the watchdog is only $3859 \mu m_2$ (0.5% of `i_croc` area), by including the wrapper it increases to $8582 \mu m_2$ (1.1% of `i_croc` area) and the power consumption is expected to remain negligible.

In addition to its primary function, we also demonstrated integration with the user domain by storing and printing our names from a ROM as demanded by the course.

5.3 Possible improvements

Although the current implementation fulfills its core objective (ensuring automatic recovery in case of system faults), several improvements could enhance usability, debuggability, and flexibility:

- **Pre-Reset Warning:** Instead of directly issuing a hard reset when the timer expires, the watchdog could first raise a pre-timeout signal. This would give the software a final chance to handle the fault (e.g., save state or attempt recovery), reducing the risk of unnecessary resets.

- **Failure Context Logging:** To aid debugging after a reset, the watchdog could store contextual information, such as the program counter (PC) or key register values, in a reserved memory location before triggering the reset. This would help developers identify the point of failure and understand why the system stalled.
- **Configurable Reset Behavior:** The current one-cycle reset is simple and effective, but offering options (e.g., soft reset, selective module reset) could improve system flexibility, especially for debugging or low-power scenarios.
- **Support for Multiple Watchdog Instances:** In multi-core or peripheral-heavy systems, separate watchdogs could be assigned to different components. This would allow more granular monitoring and reduce the risk of one component's failure affecting the entire SoC.

These improvements come at the cost of additional area and design complexity, but could be considered for future versions if more robustness or debugging capability is needed.

Chapter 6

Conclusion

6.1 Summary of key findings

Integrating our simple hardware watchdog into the CROC SoC reliably detects and recovers from processor stalls or bus deadlocks with negligible impact on area, power, or design complexity. RTL simulations, clean physical implementation, and software integration confirm that all objectives (fault coverage, automatic reset, and minimal overhead) are met.

6.2 Future Work and Next Steps

- Explore additional watchdog architectures as detailed in 5.3, selecting the most suitable type based on application requirements.
- Proceed to tape-out and fabricate our circuit :) !

Bibliography

- [1] P. Sauter, T. Benz, P. Scheffler, H. Pochert, L. Wüthrich, M. Povišer, B. Muheim, F. K. Gürkaynak, and L. Benini, “Croc: An end-to-end open-source extensible risc-v mcu platform to democratize silicon,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.05090>
- [2] Dr. David R. Williams, “Clementine Project Information,” January 2022, accessed: 2025-05-27. [Online]. Available: <https://nssdc.gsfc.nasa.gov/planetary/clementine.html>
- [3] N. Zlatanov, “Architecture and operation of a watchdog timer,” *no. July*, vol. 2014, 2014. [Online]. Available: https://www.researchgate.net/profile/Nikola-Zlatanov/publication/295010877_Architecture_and_Operation_of_a_Watchdog_Timer/links/56c65d6b08ae0d3b1b603ee0/Architecture-and-Operation-of-a-Watchdog-Timer.pdf
- [4] J. Abaffy and T. Krajčovič, “Multiple software watchdog timers in the linux os,” in *Emerging Trends in Computing, Informatics, Systems Sciences, and Engineering*, T. Sobh and K. Elleithy, Eds. New York, NY: Springer New York, 2013, pp. 759–765.